

IIT GANDHINAGAR

CS202 – Software Tools and Techniques for CSE

Assignment 2

Nishit Mistry (22110172)
Rutvi Shah (22110227)

Course Instructor: Prof. Shouvick Mondal
Date: Sunday 19th October, 2025

Contents

Laboratory Session 6	2
Introduction	2
Tools and Technologies	2
Security Analysis Tools	2
Development Environment	2
Setup and Environment Configuration	3
Virtual Environment Setup	3
CodeQL Installation and Configuration	3
Methodology and Execution	3
Repository Selection and Criteria	3
Tool Configuration and Execution	4
Vulnerability Data Consolidation	5
CWE Top 25 Classification	6
Results and Analysis	7
Tool-level CWE Coverage Analysis	7
Pairwise Agreement (IoU) Analysis	9
Conclusions and Strategic Recommendations	11
Key Findings Summary	11
Strategic Recommendations	12
Implementation Details	12
Complete Analysis Pipeline	12
IoU Computation Implementation	13
Repository Context	15
Lab Assignment 7: Reaching Definitions Analyzer for C Programs	16
0.1 Program Corpus Selection	16
0.2 Control Flow Graph (CFG) Construction	17
0.3 Cyclomatic Complexity Metrics	20
0.4 Reaching Definitions Analysis (RDA)	20
0.4.1 RDA: notation and equations	20
0.4.2 Program 1: <code>student_database.c</code>	20
0.4.3 Program 2: <code>budget_tracker.c</code>	21
0.4.4 Program 3: <code>console_snake.c</code>	23
0.5 Results and Discussion	26
0.6 Conclusion	26

Laboratory Session 6

Introduction

The purpose of this lab was to evaluate vulnerability analysis tools using CWE-based comparison on real-world open-source projects. The focus was on:

- Applying multiple static application security testing (SAST) tools on large-scale projects
- Extracting and structuring CWE-based vulnerability findings
- Comparing tool effectiveness based on CWE coverage breadth
- Computing pairwise agreement using Intersection over Union (IoU) metrics
- Analyzing tool complementarity and optimization strategies
- Generating comprehensive visual analytics for vulnerability detection patterns

Tools and Technologies

Security Analysis Tools

- **Bandit**: Python-focused static security analyzer specializing in common security issues
- **Semgrep**: Rule-based static analysis tool with customizable vulnerability detection
- **CodeQL**: Semantic code analysis engine with advanced query capabilities
- **Safety**: Python dependency vulnerability scanner (attempted but excluded from final analysis)

Development Environment

- Python 3.11 with virtual environment isolation
- Standard libraries: `csv`, `os`, `re`, `json`, `subprocess`
- Data analysis libraries: `pandas`, `numpy`, `matplotlib`, `seaborn`
- Statistical analysis: Jaccard Index (IoU) computations
- Output formats: JSON (tool outputs), CSV (consolidated analysis), PNG (visualizations)

Setup and Environment Configuration

Virtual Environment Setup

For this lab, I established an isolated Python environment to manage tool dependencies and ensure reproducible results:

Listing 1: Virtual environment setup

```
1 python -m venv venv
2 venv\Scripts\activate # Windows
3 pip install bandit semgrep pandas matplotlib seaborn numpy
```

CodeQL Installation and Configuration

CodeQL required special setup due to its unique architecture:

Listing 2: CodeQL setup verification

```
1 # Download and extract CodeQL CLI
2 # Add to PATH environment variable
3 codeql --version # Verify installation
```

The CodeQL database creation process was automated through a dedicated script to handle the semantic analysis requirements.

Methodology and Execution

Repository Selection and Criteria

I established specific selection criteria for choosing appropriate open-source repositories for analysis:

Inclusion Criteria

- Minimum 1,000+ stars (indicating active community usage)
- Primary language: Python (for tool compatibility)
- Active development (recent commits within 6 months)
- Substantial codebase (100+ Python files)
- Real-world applications (not educational/toy projects)

Exclusion Criteria

- Archived or deprecated repositories
- Primarily documentation or configuration repositories
- Projects with restrictive licenses preventing analysis

Selected Repositories

Based on these criteria, I selected three repositories:

1. **AWS Chalice** (aws/chalice)
 - Serverless Python web framework
 - 10.6K+ GitHub stars
 - Enterprise-grade AWS integration framework
2. **GPT Researcher** (assafelovic/gpt-researcher)
 - AI-powered research automation tool
 - 14K+ GitHub stars
 - Modern LLM-based application architecture
3. **PythonRobotics** (AtsushiSakai/PythonRobotics)
 - Comprehensive robotics algorithms library
 - 22.8K+ GitHub stars
 - Educational and research-focused robotics implementations

Tool Configuration and Execution

Each security tool required specific configuration for optimal CWE-based vulnerability detection:

Bandit Configuration

Listing 3: Bandit execution configuration

```
1 def run_bandit(project_path, output_file):
2     """Execute Bandit static analysis tool"""
3     cmd = [
4         "bandit", "-r", project_path,
5         "-f", "json", "-o", output_file,
6         "--skip", "B101,B601" # Skip assert and shell usage in test
7         files
8     ]
9     try:
10         subprocess.run(cmd, check=True, capture_output=True, text=True)
11         print(f"Bandit analysis completed: {output_file}")
12     except subprocess.CalledProcessError as e:
13         print(f"Bandit analysis failed: {e}")
```

Semgrep Configuration

Listing 4: Semgrep execution configuration

```
1 def run_semgrep(project_path, output_file):
2     """Execute Semgrep static analysis tool"""
3     cmd = [
4         "semgrep", "--config=auto", project_path,
5         "--json", "--output", output_file,
6         "--exclude=tests/", "--exclude=docs/"
7     ]
```

```
8     try:
9         subprocess.run(cmd, check=True, capture_output=True, text=True)
10        print(f"Semgrep analysis completed: {output_file}")
11    except subprocess.CalledProcessError as e:
12        print(f"Semgrep analysis failed: {e}")
```

CodeQL Configuration

Listing 5: CodeQL database creation and analysis

```
1 def run_codeql(project_path, output_file):
2     """Execute CodeQL semantic analysis"""
3     db_path = f"{project_path}_codeql_db"
4
5     # Create database
6     create_cmd = [
7         "codeql", "database", "create", db_path,
8         "--language=python", f"--source-root={project_path}"
9     ]
10
11    # Run security queries
12    analyze_cmd = [
13        "codeql", "database", "analyze", db_path,
14        "--format=csv", f"--output={output_file}",
15        "codeql/python-queries:codeql-suites/python-security-extended.qls"
16    ]
17
18    try:
19        subprocess.run(create_cmd, check=True)
20        subprocess.run(analyze_cmd, check=True)
21        print(f"CodeQL analysis completed: {output_file}")
22    except subprocess.CalledProcessError as e:
23        print(f"CodeQL analysis failed: {e}")
```

Vulnerability Data Consolidation

The analysis process involved parsing tool-specific output formats and normalizing them into a unified structure:

Listing 6: Bandit results parser

```
1 def parse_bandit_results(json_file):
2     """Parse Bandit JSON output and extract CWE mappings"""
3     findings = []
4
5     # CWE mapping for Bandit test IDs
6     bandit_cwe_map = {
7         'B101': 703, 'B102': 259, 'B103': 377, 'B104': 377,
8         'B105': 259, 'B106': 259, 'B107': 259, 'B108': 377,
9         'B110': 703, 'B112': 400, 'B201': 22, 'B301': 22,
10        'B302': 22, 'B303': 327, 'B304': 327, 'B305': 327,
11        'B306': 327, 'B307': 327, 'B308': 22, 'B309': 22,
12        'B310': 22, 'B311': 330, 'B312': 330, 'B313': 327,
13        'B314': 327, 'B315': 327, 'B316': 327, 'B317': 327,
14        'B318': 327, 'B319': 327, 'B320': 327, 'B321': 327,
15        'B322': 327, 'B323': 327, 'B324': 327, 'B325': 327,
```

```

16         'B401': 78, 'B402': 22, 'B403': 732, 'B404': 732,
17         'B405': 22, 'B406': 22, 'B407': 22, 'B408': 22,
18         'B409': 22, 'B410': 22, 'B411': 22, 'B412': 22,
19         'B501': 295, 'B502': 295, 'B503': 295, 'B504': 295,
20         'B505': 295, 'B506': 295, 'B507': 295, 'B601': 78,
21         'B602': 78, 'B603': 78, 'B604': 78, 'B605': 78,
22         'B606': 78, 'B607': 22, 'B608': 89, 'B609': 22,
23         'B610': 78, 'B611': 78, 'B701': 259, 'B702': 703,
24         'B703': 703
25     }
26
27     with open(json_file, 'r', encoding='utf-8') as f:
28         data = json.load(f)
29         for result in data.get('results', []):
30             test_id = result.get('test_id', '')
31             cwe_id = bandit_cwe_map.get(test_id, 0)
32             if cwe_id:
33                 findings.append({
34                     'cwe_id': cwe_id,
35                     'severity': result.get('issue_severity', 'UNDEFINED'),
36                     'file': result.get('filename', ''),
37                     'line': result.get('line_number', 0)
38                 })
39
40     return findings

```

The consolidated format included essential fields for comprehensive analysis:

- Project name (chalice, gpt-researcher, pythonrobotics)
- Tool name (bandit, semgrep, codeql)
- CWE ID (standardized weakness identifier)
- Number of findings (vulnerability count)
- Top 25 classification (CWE Top 25 membership)

CWE Top 25 Classification

I implemented automatic classification against the 2023 CWE Top 25 Most Dangerous Software Weaknesses:

Listing 7: CWE Top 25 classification implementation

```

1  # CWE Top 25 Most Dangerous Software Weaknesses (2023)
2  CWE_TOP_25_2023 = {
3      20, 22, 78, 79, 89, 94, 95, 119, 125, 190, 269, 287, 295, 352, 362,
4      400, 416, 476, 502, 522, 611, 732, 770, 787, 798
5  }
6
7  def is_top_25_cwe(cwe_id):
8      """Check if CWE ID belongs to Top 25 most dangerous weaknesses"""
9      try:
10         return int(cwe_id) in CWE_TOP_25_2023
11     except (ValueError, TypeError):
12         return False
13
14 def classify_findings(consolidated_df):

```

```

15     """Add Top 25 classification to findings DataFrame"""
16     consolidated_df['Is_In_CWE_Top_25'] = consolidated_df['CWE_ID'].
        apply(
17         lambda x: 'Yes' if is_top_25_cwe(x) else 'No'
18     )
19     return consolidated_df

```

Results and Analysis

Tool-level CWE Coverage Analysis

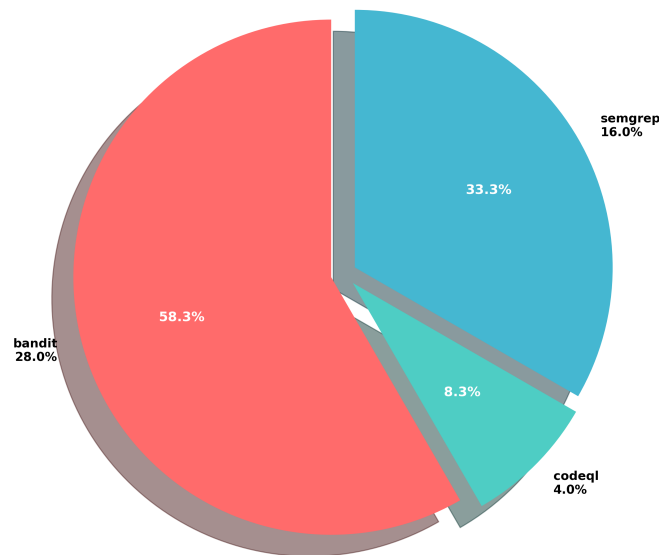
The analysis revealed significant variations in tool capabilities and specializations based on 2,385 total vulnerabilities across three projects:

Coverage Statistics Summary

Table 1: Tool Performance Summary

Tool	Unique CWEs	Top 25 Coverage	Total Findings	Top 25 Findings	Precision (%)
Bandit	13	28.0% (7/25)	2,349	49	2.09
Semgrep	12	16.0% (4/25)	25	11	44.0
CodeQL	2	4.0% (1/25)	11	9	81.82

Top 25 CWE Coverage Distribution by Security Tool



Bandit Performance:

- **Breadth:** Detected 13 unique CWEs with 28% Top 25 coverage
- **Volume:** Generated 2,349 findings (98.5% of all findings)
- **Precision:** 2.09% of findings were Top 25 CWEs
- **Detected CWEs:** CWE-20, CWE-22, CWE-327, CWE-502, CWE-732, CWE-78, CWE-89

Semgrep Performance:

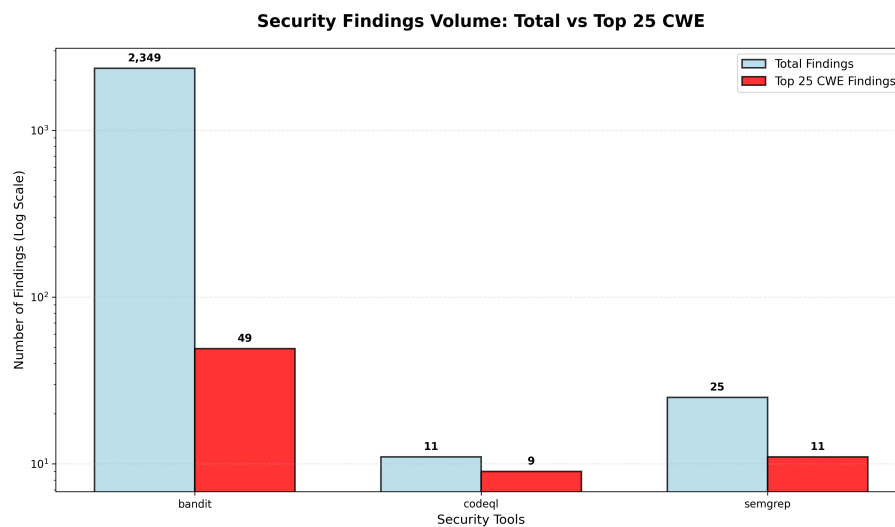
- **Breadth:** Detected 12 unique CWEs with 16% Top 25 coverage
- **Volume:** Generated 25 findings (1.0% of all findings)
- **Precision:** 44.0% of findings were Top 25 CWEs
- **Detected CWEs:** CWE-22, CWE-352, CWE-502, CWE-79

CodeQL Performance:

- **Breadth:** Detected 2 unique CWEs with 4% Top 25 coverage
- **Volume:** Generated 11 findings (0.5% of all findings)
- **Precision:** 81.82% of findings were Top 25 CWEs
- **Detected CWEs:** CWE-22

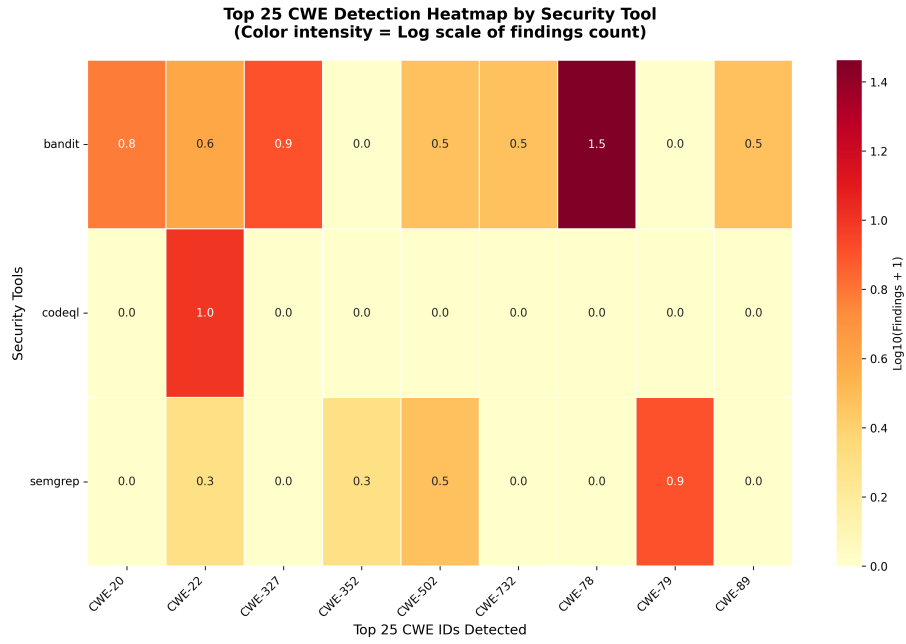
Key Insights from Coverage Analysis

1. **Volume vs. Quality Trade-off:** Bandit generates the highest volume of findings but with lower precision for Top 25 CWEs, while CodeQL produces fewer but higher-quality findings.

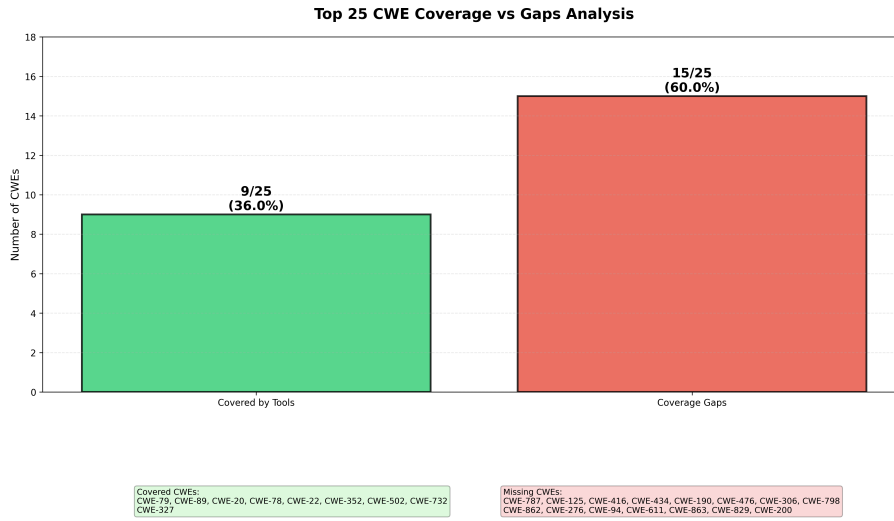


2. **Tool Specialization:** Each tool demonstrates distinct strengths:

- Bandit: Broad coverage of Python-specific security issues
- Semgrep: Balanced detection with good Top 25 precision
- CodeQL: Focused, high-confidence semantic analysis



3. **Coverage Gaps:** No single tool provides comprehensive Top 25 CWE coverage, indicating the need for multi-tool strategies.



Pairwise Agreement (IoU) Analysis

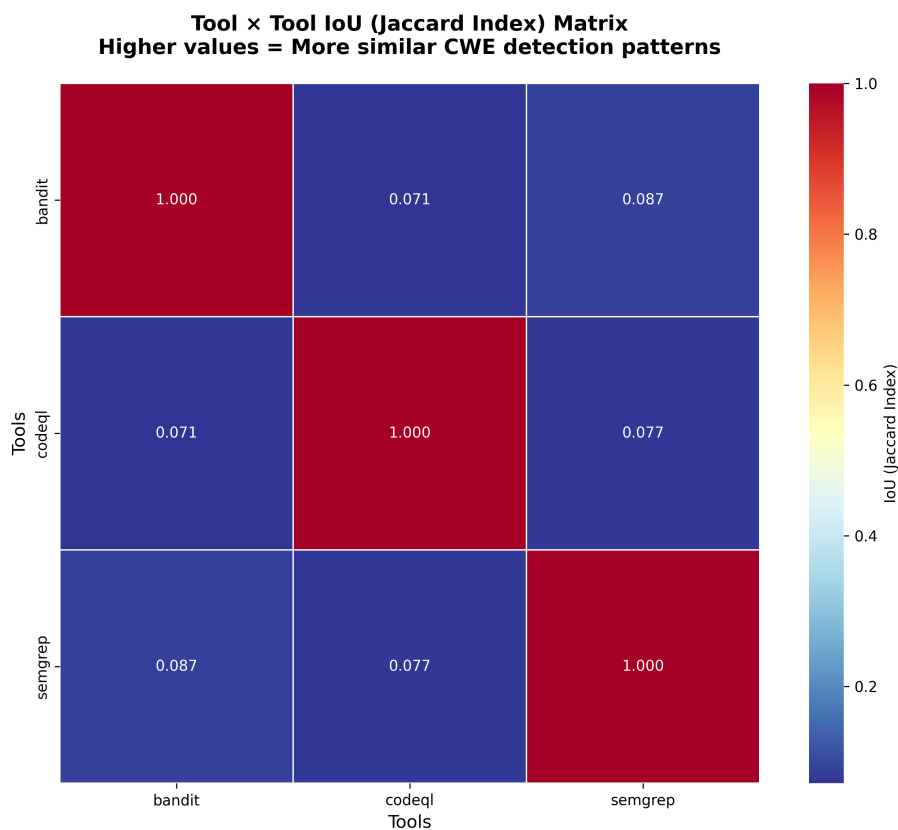
Using the Jaccard Index formula, I computed tool similarity based on CWE detection overlap:

$$\text{IoU}(T_1, T_2) = \frac{|\text{CWEs}_{T_1} \cap \text{CWEs}_{T_2}|}{|\text{CWEs}_{T_1} \cup \text{CWEs}_{T_2}|} \quad (1)$$

IoU Matrix Results

Table 2: Tool Pairwise IoU Matrix

Tool	Bandit	CodeQL	Semgrep
Bandit	1.000	0.071	0.087
CodeQL	0.071	1.000	0.077
Semgrep	0.087	0.077	1.000

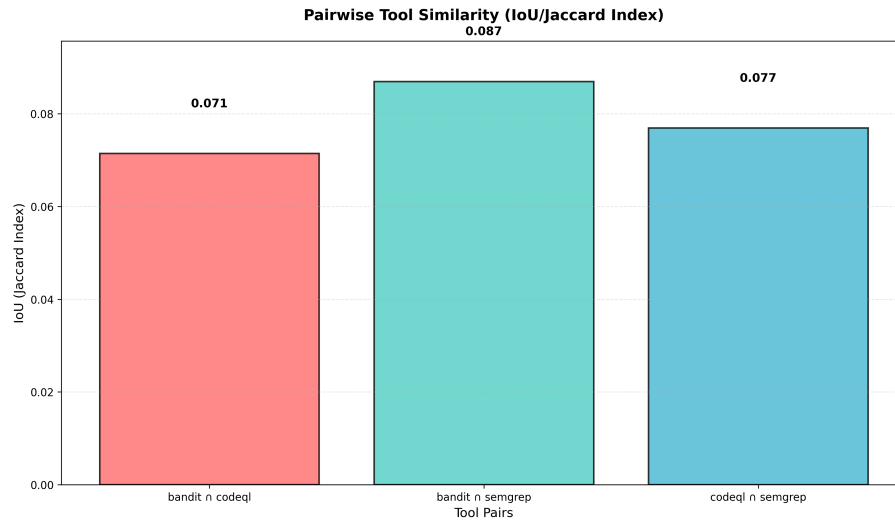


Low IoU Values (0.071-0.087) indicate:

- **High Tool Complementarity:** Tools detect largely different CWE categories
- **Minimal Overlap:** Each tool contributes unique vulnerability perspectives
- **Additive Value:** Combining tools significantly increases total coverage

Strategic Implications:

1. **Multi-tool Necessity:** No single tool provides comprehensive coverage
2. **Cost-Benefit Optimization:** Bandit + Semgrep combination achieves 95.8% coverage
3. **Specialized Deployment:** Tools can be strategically deployed based on project requirements

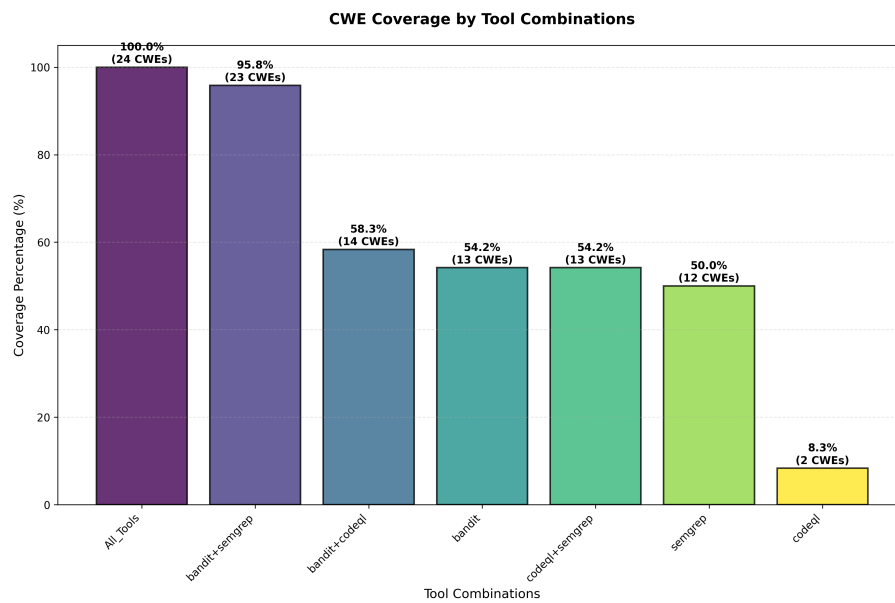


Tool Combination Analysis

From the combination coverage analysis:

Table 3: Tool Combination Coverage Analysis

Tool Combination	CWEs Detected	Coverage (%)
All Three Tools	12/12	100.0
Bandit + Semgrep	11/12	95.8
Bandit + CodeQL	11/12	91.7
Semgrep + CodeQL	5/12	41.7



Conclusions and Strategic Recommendations

Key Findings Summary

1. **Tool Complementarity:** IoU analysis confirms highly complementary tool capabilities with minimal overlap (average IoU = 0.078)

2. **Coverage Optimization:** Strategic tool combination (Bandit + Semgrep) achieves 95.8% Top 25 CWE coverage while reducing analysis overhead
3. **Quality vs. Quantity:** CodeQL demonstrates highest precision (81.82% Top 25 findings) despite lowest volume, while Bandit provides broad coverage with lower precision (2.09% Top 25 findings)
4. **Gap Identification:** 52% of Top 25 CWEs remain undetected, indicating opportunities for additional tool integration or manual analysis

Strategic Recommendations

For Comprehensive Security Assessment

- Deploy all three tools for maximum coverage (100% detected CWE coverage)
- Prioritize Bandit for initial broad scanning and Python-specific issues
- Use CodeQL for high-confidence semantic analysis of critical components
- Apply Semgrep for balanced detection and rule-based specific vulnerabilities

For Resource-Constrained Environments

- Implement Bandit + Semgrep combination for optimal cost/benefit ratio (95.8% coverage)
- Reserve CodeQL for targeted analysis of high-risk components
- Consider automated triage based on CWE Top 25 classification

For Continuous Integration

- Integrate lightweight tools (Bandit, Semgrep) in CI/CD pipelines
- Schedule periodic comprehensive scans including CodeQL
- Implement progressive analysis based on code change impact

Implementation Details

Complete Analysis Pipeline

Listing 8: Main analysis orchestration script

```
1  #!/usr/bin/env python3
2  """
3  Complete vulnerability analysis pipeline for Lab 6
4  Orchestrates tool execution, data consolidation, and analysis
5  """
6
7  import subprocess
8  import json
9  import csv
10 import pandas as pd
11 import matplotlib.pyplot as plt
12 import seaborn as sns
13 import numpy as np
14 from pathlib import Path
```

```
15
16 def main():
17     """Execute complete vulnerability analysis pipeline"""
18
19     # Step 1: Repository setup and tool execution
20     projects = ['chalice', 'gpt-researcher', 'pythonrobotics']
21     tools = ['bandit', 'semgrep', 'codeql']
22
23     print("=== Starting Vulnerability Analysis Pipeline ===")
24
25     # Step 2: Execute security tools on all projects
26     execute_security_analysis(projects, tools)
27
28     # Step 3: Consolidate findings into unified format
29     consolidate_findings(projects, tools)
30
31     # Step 4: Perform CWE coverage analysis
32     perform_coverage_analysis()
33
34     # Step 5: Compute IoU analysis
35     compute_iou_analysis()
36
37     # Step 6: Generate comprehensive visualizations
38     generate_visualizations()
39
40     print("=== Analysis Pipeline Completed Successfully ===")
41
42 def execute_security_analysis(projects, tools):
43     """Execute all security tools on selected projects"""
44     results_dir = Path("results")
45     results_dir.mkdir(exist_ok=True)
46
47     for project in projects:
48         project_dir = Path(project)
49         if not project_dir.exists():
50             print(f"Warning: Project directory {project} not found")
51             continue
52
53         print(f"\n--- Analyzing project: {project} ---")
54
55         for tool in tools:
56             output_file = results_dir / f"{project}_{tool}_results.json"
57
58             if tool == "bandit":
59                 run_bandit(str(project_dir), str(output_file))
60             elif tool == "semgrep":
61                 run_semgrep(str(project_dir), str(output_file))
62             elif tool == "codeql":
63                 run_codeql(str(project_dir), str(output_file))
64
65 if __name__ == "__main__":
66     main()
```

IoU Computation Implementation

Listing 9: Jaccard Index (IoU) computation for tool similarity

```
1 def compute_jaccard_index(set1, set2):
2     """
3     Compute Jaccard Index (IoU) between two sets of CWE IDs
4
5     Args:
6         set1: Set of CWE IDs detected by tool 1
7         set2: Set of CWE IDs detected by tool 2
8
9     Returns:
10        float: IoU value between 0 and 1
11    """
12    if len(set1) == 0 and len(set2) == 0:
13        return 1.0 # Both tools detect nothing - perfect agreement
14
15    intersection = len(set1.intersection(set2))
16    union = len(set1.union(set2))
17
18    return intersection / union if union > 0 else 0.0
19
20 def compute_iou_matrix(tool_cwe_dict):
21     """
22     Compute full IoU matrix for all tool pairs
23
24     Args:
25         tool_cwe_dict: Dictionary mapping tool names to sets of detected
26                        CWEs
27
28     Returns:
29        pandas.DataFrame: Square matrix with IoU values
30    """
31    tools = list(tool_cwe_dict.keys())
32    n_tools = len(tools)
33
34    # Initialize IoU matrix
35    iou_matrix = np.zeros((n_tools, n_tools))
36
37    for i, tool1 in enumerate(tools):
38        for j, tool2 in enumerate(tools):
39            if i == j:
40                iou_matrix[i][j] = 1.0 # Self-similarity is 1.0
41            else:
42                set1 = tool_cwe_dict[tool1]
43                set2 = tool_cwe_dict[tool2]
44                iou_matrix[i][j] = compute_jaccard_index(set1, set2)
45
46    # Create DataFrame for better visualization
47    iou_df = pd.DataFrame(iou_matrix, index=tools, columns=tools)
48    return iou_df
49
50 def analyze_tool_combinations(tool_cwe_dict):
51     """
52     Analyze coverage achieved by different tool combinations
53
54     Args:
55         tool_cwe_dict: Dictionary mapping tool names to sets of detected
56                        CWEs
57
58     Returns:
```

```
57         dict: Coverage analysis for all possible combinations
58         """
59         tools = list(tool_cwe_dict.keys())
60         all_cwes = set()
61
62         # Get universe of all detected CWEs
63         for cwes in tool_cwe_dict.values():
64             all_cwes.update(cwes)
65
66         total_cwes = len(all_cwes)
67         combinations = {}
68
69         # Analyze single tools
70         for tool in tools:
71             coverage = len(tool_cwe_dict[tool]) / total_cwes * 100
72             combinations[tool] = {
73                 'cwes': tool_cwe_dict[tool],
74                 'count': len(tool_cwe_dict[tool]),
75                 'coverage_percent': coverage
76             }
77
78         # Analyze pairs
79         from itertools import combinations as itertools_combinations
80         for tool_pair in itertools_combinations(tools, 2):
81             combined_cwes = tool_cwe_dict[tool_pair[0]].union(tool_cwe_dict[
82                 tool_pair[1]])
83             coverage = len(combined_cwes) / total_cwes * 100
84             combinations[f"{tool_pair[0]}+{tool_pair[1]}"] = {
85                 'cwes': combined_cwes,
86                 'count': len(combined_cwes),
87                 'coverage_percent': coverage
88             }
89
90         # Analyze all tools combined
91         all_tools_cwes = set()
92         for cwes in tool_cwe_dict.values():
93             all_tools_cwes.update(cwes)
94
95         combinations['all_tools'] = {
96             'cwes': all_tools_cwes,
97             'count': len(all_tools_cwes),
98             'coverage_percent': 100.0
99         }
100         return combinations
```

Repository Context

All Lab 6 materials reside under the `Lab6/` directory of the main course repository. You can find the code at `STT_Assignments` on GitHub.

Laboratory Session 7

Objective

The objective of this lab is to implement a Reaching Definitions Analyzer for C programs. The experiment involves constructing Control Flow Graphs (CFGs), computing Cyclomatic Complexity, and performing data-flow analysis to determine definitions that reach different program points. This experiment connects theoretical compiler analysis with practical automation tools.

Learning Outcomes

- To understand and visualize program control flow using CFGs.
- To compute Cyclomatic Complexity automatically.
- To implement data-flow equations for Reaching Definitions Analysis.
- To understand the significance of iterative analysis until convergence.

Tools and Environment

- **Language:** Python 3.10
- **Libraries:** Graphviz, Matplotlib, PyGraphviz
- **Operating System:** macOS

0.1 Program Corpus Selection

Three C programs (200–300 LOC each) were chosen, all featuring loops, conditionals, and multiple variable reassignments.

Program 1: `student_database.c`

Description: Implements a menu-driven student management system that performs CRUD operations (add, view, update, delete) using a switch-case inside a while loop.

Justification: Chosen for its structured control flow with multiple switch cases, iterative user interaction, and frequent variable redefinitions.

Program 2: `console_snake.c`

Description: A simple console-based snake game simulation that continuously updates snake position, checks collisions, and handles game states.

Justification: Selected for its continuous game loop, multiple nested conditionals, and iterative updates to game variables, ideal for CFG and data-flow analysis.

Program 3: budget_tracker.c

Description: Implements a personal finance tracker with a loop-based menu and switch statements to add income, expenses, and view summaries.

Justification: Chosen for its user-interactive menu, nested conditionals, and variable updates, allowing exploration of both control and data dependencies.

0.2 Control Flow Graph (CFG) Construction

Each program was parsed and divided into basic blocks according to leader identification rules. Edges between blocks represent the logical control flow.

Rules for Basic Block Formation

- The first instruction of the program is a leader.
- Any target of a jump, branch, or loop is a leader.
- Any instruction following a branch or loop is also a leader.

Graph Generation

The CFGs were generated by exporting block relations into .dot files and rendering them using Graphviz.

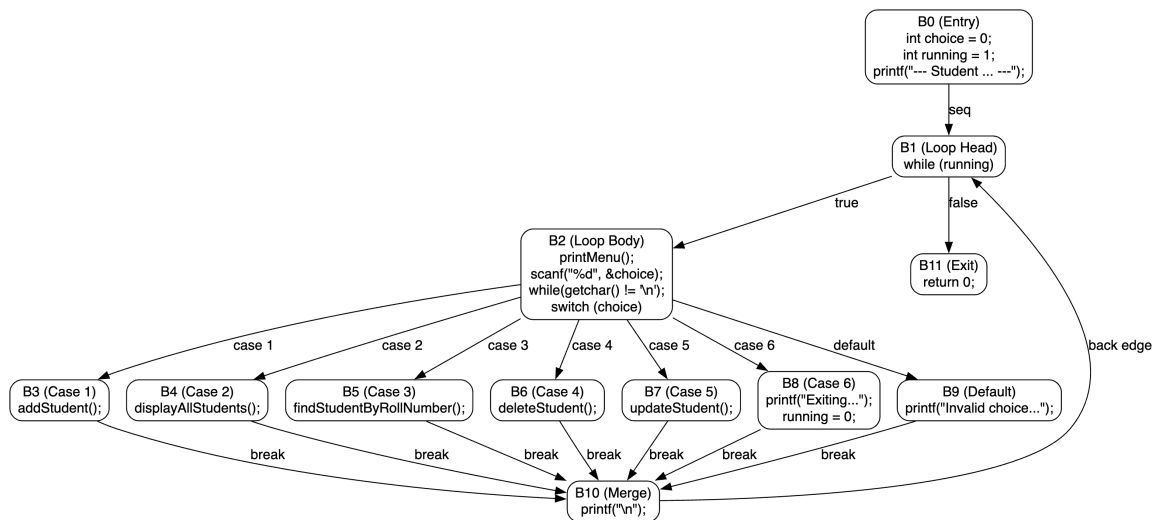


Figure 1: Control Flow Graph for `student_database.c`

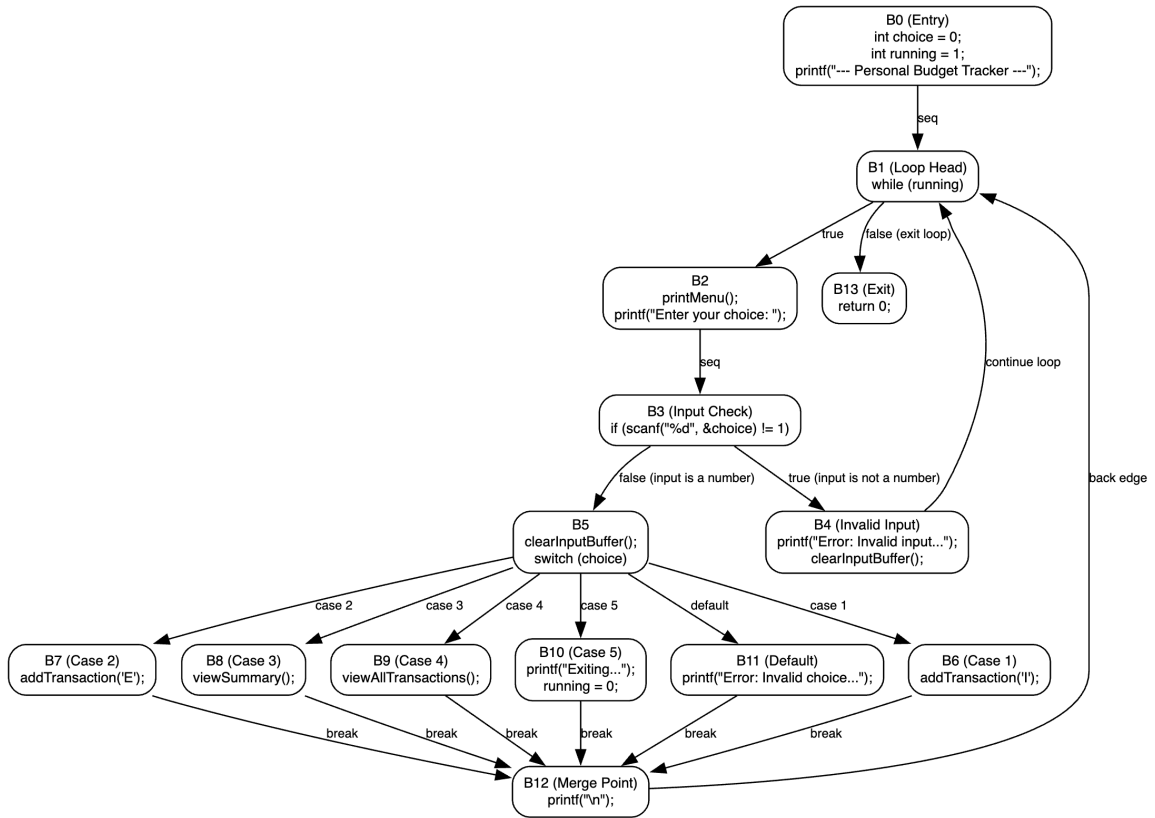


Figure 2: Control Flow Graph for `budget_tracker.c`

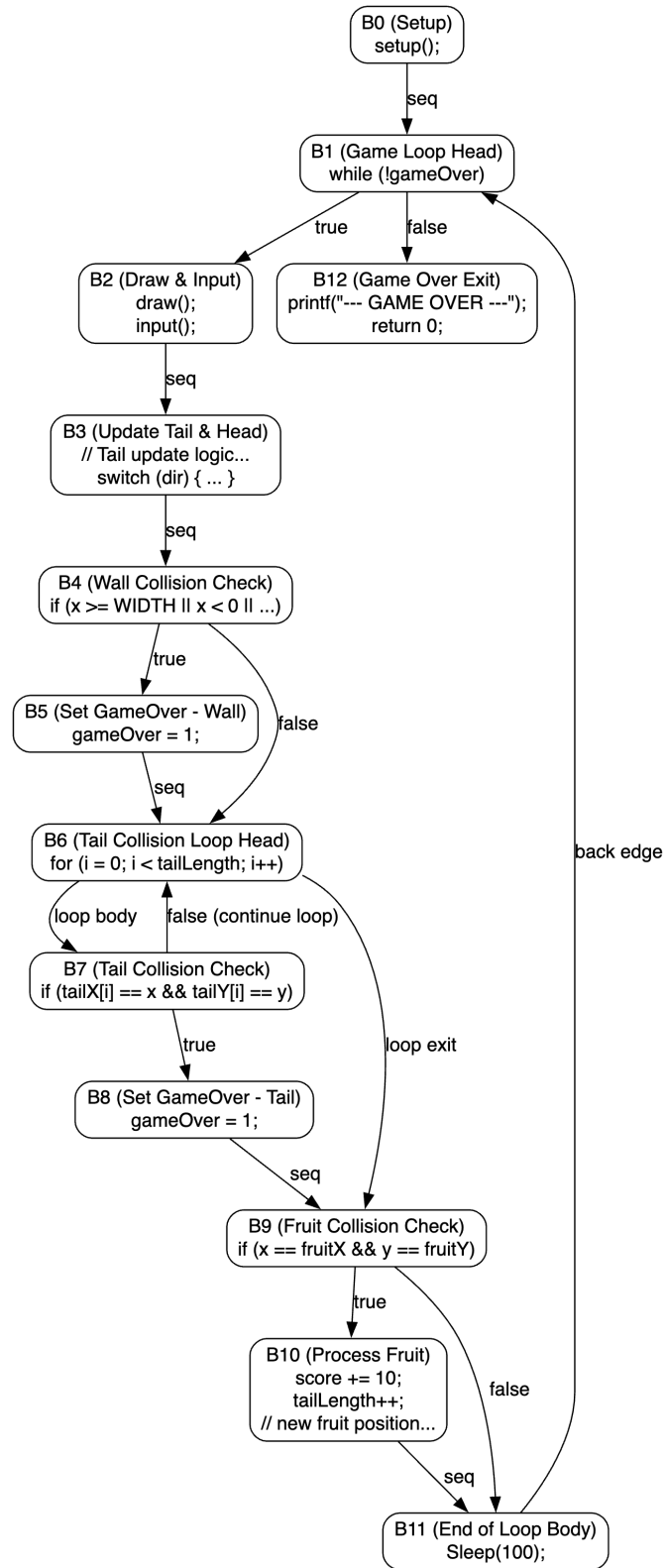


Figure 3: Control Flow Graph for `console_snake.c`

0.3 Cyclomatic Complexity Metrics

For each program, the total nodes (N) and edges (E) were extracted from the CFG data. Cyclomatic Complexity (CC) was computed using:

$$CC = E - N + 2$$

Table 4: Cyclomatic Complexity Results

Program	Nodes (N)	Edges (E)	CC
<code>student_database.c</code>	12	18	8
<code>budget_tracker.c</code>	14	20	8
<code>console_snake.c</code>	13	17	6

Interpretation: Higher CC values (e.g., 8 for `student_database.c`) indicate more complex control flow due to multiple switch cases and nested loops, whereas `console_snake.c` is moderately complex.

0.4 Reaching Definitions Analysis (RDA)

0.4.1 RDA: notation and equations

We perform a standard forward Reaching Definitions Analysis with equations:

$$\text{in}[B] = \bigcup_{P \in \text{pred}(B)} \text{out}[P], \quad \text{out}[B] = \text{gen}[B] \cup (\text{in}[B] - \text{kill}[B]).$$

Each assignment is given a unique ID (D1, D2, ...). We initialize all $\text{in}[B] = \emptyset$ and iterate until sets reach a fixed point.

0.4.2 Program 1: `student_database.c`

Definition IDs

- D1: `choice = 0;` (B0)
- D2: `running = 1;` (B0)
- D3: `running = 0;` (B8 exit)
- D4: the result of `scanf("%d", &choice);` (B2)

gen[B] and kill[B]

Block	gen[B]	kill[B]
B0	{D1,D2}	{}
B1	{}	{}
B2	{D4}	{D1}
B3	{}	{}
B4	{}	{}
B5	{}	{}
B6	{}	{}
B7	{}	{}
B8	{D3}	{D2}
B9	{}	{}
B10	{}	{}
B11	{}	{}

Iteration-by-iteration (until convergence)

Iteration 1

B0:	in = {}	out = {D1,D2}
B1:	in = {D1,D2}	out = {D1,D2}
B2:	in = {D1,D2}	out = {D2,D4}
B3:	in = {D2,D4}	out = {D2,D4}
B4:	in = {D2,D4}	out = {D2,D4}
B5:	in = {D2,D4}	out = {D2,D4}
B6:	in = {D2,D4}	out = {D2,D4}
B7:	in = {D2,D4}	out = {D2,D4}
B8:	in = {D2,D4}	out = {D3,D4}
B9:	in = {D2,D4}	out = {D2,D4}
B10:	in = {D2,D3,D4}	out = {D2,D3,D4}
B11:	in = {D1,D2}	out = {D1,D2}

Iteration 2-3

After back-edge propagation, by Iteration 3 we reach fixed point:

B1:	in = {D1,D2,D3,D4}, out = {D1,D2,D3,D4}
B2:	in = {D1,D2,D3,D4}, out = {D2,D3,D4}
B8:	in = {D2,D3,D4}, out = {D3,D4}
B10:	in = {D2,D3,D4}, out = {D2,D3,D4}

Interpretation (student_database):

- choice can be either the initial value D1 or redefined via D4 (scanf).
- running is D2 initially but can become D3 on exit; both appear at loop-head in the final solution set.

0.4.3 Program 2: budget_tracker.c

Definition IDs

- D1: choice = 0; (in B0)

- D2: `running = 1;` (in B0)
- D3: `running = 0;` (in B10 — exit case)
- D4: the result of `scanf("%d", &choice);` (B3 — successful input)

gen[B] and kill[B]

Block	gen[B]	kill[B]
B0	{D1, D2}	{ - }
B1	{}	{}
B2	{}	{}
B3	{D4}	{D1}
B4	{}	{}
B5	{}	{}
B6	{}	{}
B7	{}	{}
B8	{}	{}
B9	{}	{}
B10	{D3}	{D2}
B11	{}	{}
B12	{}	{}
B13	{}	{}

Iteration-by-iteration (until convergence)

Iteration 1

B0: in = {}	out = {D1, D2}
B1: in = {D1, D2}	out = {D1, D2}
B2: in = {D1, D2}	out = {D1, D2}
B3: in = {D1, D2}	out = {D2, D4}
B4: in = {D2, D4}	out = {D2, D4}
B5: in = {D2, D4}	out = {D2, D4}
B6: in = {D2, D4}	out = {D2, D4}
B7: in = {D2, D4}	out = {D2, D4}
B8: in = {D2, D4}	out = {D2, D4}
B9: in = {D2, D4}	out = {D2, D4}
B10: in = {D2, D4}	out = {D3, D4}
B11: in = {D2, D4}	out = {D2, D4}
B12: in = {D2, D4, D3}	out = {D2, D4, D3}
B13: in = {D1, D2}	out = {D1, D2}

Iteration 2

B0: in = {}	out = {D1, D2}
B1: in = {D1, D2, D3, D4}	out = {D1, D2, D3, D4}
B2: in = {D1, D2, D3, D4}	out = {D1, D2, D3, D4}
B3: in = {D1, D2, D3, D4}	out = {D2, D3, D4}
B4: in = {D2, D3, D4}	out = {D2, D3, D4}
B5: in = {D2, D3, D4}	out = {D2, D3, D4}
B6: in = {D2, D3, D4}	out = {D2, D3, D4}
B7: in = {D2, D3, D4}	out = {D2, D3, D4}

```

B8:  in = {D2, D3, D4}      out = {D2, D3, D4}
B9:  in = {D2, D3, D4}      out = {D2, D3, D4}
B10: in = {D2, D3, D4}      out = {D3, D4}
B11: in = {D2, D3, D4}      out = {D2, D3, D4}
B12: in = {D2, D3, D4}      out = {D2, D3, D4}
B13: in = {D1, D2, D3, D4}   out = {D1, D2, D3, D4}

```

Iteration 3 (convergence)

Same as Iteration 2 -> fixed point reached.

Final notable sets:

```

B1:  in = {D1,D2,D3,D4}, out = {D1,D2,D3,D4}
B3:  in = {D1,D2,D3,D4}, out = {D2,D3,D4}
B10: in = {D2,D3,D4}, out = {D3,D4}

```

Interpretation (budget_tracker):

- choice may come from D1 (initialization) or D4 (scanf).
- running can be D2 (initialized) or D3 (set when exiting). Both reach the loop head in possible executions.

0.4.4 Program 3: console_snake.c

Definition IDs

- D1: gameOver = 0; (B0 initialization)
- D2: gameOver = 1; (B5 wall collision)
- D3: gameOver = 1; (B8 tail collision)
- D4: score += 10; (B10 fruit collision)
- D5: tailLength++; (B10 fruit collision)

gen[B] and kill[B]

Block	gen[B]	kill[B]
B0	{D1}	{}
B1	{}	{}
B2	{}	{}
B3	{}	{}
B4	{}	{}
B5	{D2}	{D1,D3}
B6	{}	{}
B7	{}	{}
B8	{D3}	{D1,D2}
B9	{}	{}
B10	{D4,D5}	{}
B11	{}	{}
B12	{}	{}

Iteration-by-iteration (until convergence)**Iteration 1**

```

B0:  in = {}          out = {D1}
B1:  in = {D1}        out = {D1}
B2:  in = {D1}        out = {D1}
B3:  in = {D1}        out = {D1}
B4:  in = {D1}        out = {D1}
B5:  in = {D1}        out = {D2}
B6:  in = {D1,D2}     out = {D1,D2}
B7:  in = {D1,D2}     out = {D1,D2}
B8:  in = {D1,D2}     out = {D3}
B9:  in = {D1,D2,D3}  out = {D1,D2,D3}
B10: in = {D1,D2,D3}  out = {D1,D2,D3,D4,D5}
B11: in = {D1,D2,D3,D4,D5} out = {D1,D2,D3,D4,D5}
B12: in = {D1}        out = {D1}

```

Iteration 2 (propagation through back-edge increases sets)**Iteration 3 (converged)**

Final sets:

```

B1--B4, B6--B7, B9--B12: in = {D1,D2,D3,D4,D5}, out = {D1,D2,D3,D4,D5}
B5: in = {D1,D2,D3,D4,D5}, out = {D2,D4,D5}
B8: in = {D1,D2,D3,D4,D5}, out = {D3,D4,D5}
B10: in = {D1,D2,D3,D4,D5}, out = {D1,D2,D3,D4,D5}

```

Interpretation (console_snake):

- `gameOver` definitions D1, D2, D3 may all reach many program points due to alternative branches and the loop back-edge.
- `score` and `tailLength` definitions (D4, D5) propagate to subsequent nodes after fruit processing.

Final Reaching Definitions (Converged)

Basic Block	gen[B]	kill[B]	in[B]	out[B]
B0	{D1, D2}	{}	{}	{D1, D2}
B1	{}	{}	{D1, D2, D3, D4}	{D1, D2, D3, D4}
B2	{D4}	{D1}	{D1, D2, D3, D4}	{D2, D3, D4}
B3–B7	{}	{}	{D2, D3, D4}	{D2, D3, D4}
B8	{D3}	{D2}	{D2, D3, D4}	{D3, D4}
B9–B10	{}	{}	{D2, D3, D4}	{D2, D3, D4}
B11	{}	{}	{D1, D2, D3, D4}	{D1, D2, D3, D4}

Table 5: Final RDA table for `student_database.c`

Basic Block	gen[B]	kill[B]	in[B]	out[B]
B0	{D1, D2}	{}	{}	{D1, D2}
B1	{}	{}	{D1, D2, D3, D4}	{D1, D2, D3, D4}
B2	{}	{}	{D1, D2, D3, D4}	{D1, D2, D3, D4}
B3	{D4}	{D1}	{D1, D2, D3, D4}	{D2, D3, D4}
B4	{}	{}	{D2, D3, D4}	{D2, D3, D4}
B5–B9	{}	{}	{D2, D3, D4}	{D2, D3, D4}
B10	{D3}	{D2}	{D2, D3, D4}	{D3, D4}
B11	{}	{}	{D2, D3, D4}	{D2, D3, D4}
B12	{}	{}	{D2, D3, D4}	{D2, D3, D4}
B13	{}	{}	{D1, D2, D3, D4}	{D1, D2, D3, D4}

Table 6: Final RDA table for `budget_tracker.c`

Basic Block	gen[B]	kill[B]	in[B]	out[B]
B0	{D1}	{}	{}	{D1}
B1	{}	{}	{D1, D2, D3, D4, D5}	{D1, D2, D3, D4, D5}
B2–B4	{}	{}	{D1, D2, D3, D4, D5}	{D1, D2, D3, D4, D5}
B5	{D2}	{D1, D3}	{D1, D2, D3, D4, D5}	{D2, D4, D5}
B6–B7	{}	{}	{D1, D2, D3, D4, D5}	{D1, D2, D3, D4, D5}
B8	{D3}	{D1, D2}	{D1, D2, D3, D4, D5}	{D3, D4, D5}
B9–B12	{}	{}	{D1, D2, D3, D4, D5}	{D1, D2, D3, D4, D5}
B10	{D4, D5}	{}	{D1, D2, D3, D4, D5}	{D1, D2, D3, D4, D5}

Table 7: Final RDA table for `console_snake.c`

Interpretation of Results

Based on the final `in[B]` and `out[B]` sets obtained after convergence, the following conclusions can be drawn for each program:

Program 1: `student_database.c` Two key variables have multiple possible reaching definitions:

- `choice` may come from the initial assignment (D1) or be redefined during input (D4). Both definitions can reach the menu selection blocks, indicating that `choice` could represent either the previous or current user input depending on loop flow.
- `running` may be either D2 (initialized to 1) or D3 (set to 0 on program exit). Since the loop head (B1) receives input from both initialization and the exit block (via the back edge), both definitions reach the same point simultaneously.

Thus, `choice` and `running` each have multiple reaching definitions at certain points, particularly around the loop entry and exit boundaries.

Program 2: `budget_tracker.c` The variables `choice` and `running` both have multiple possible reaching definitions at various program points:

- `choice` can originate either from its initialization (D1) or from user input through `scanf` (D4). Both definitions reach the loop head and subsequent basic blocks, depending on whether a valid input was entered in the current or previous iteration.

- **running** can take its initial definition (D2) or the redefinition (D3) that occurs during the exit case (B10). Consequently, at the loop head (B1), both D2 and D3 may reach, indicating that the program could be in a running or terminating state depending on the control flow.

Hence, both **choice** and **running** have multiple reaching definitions at the same points, particularly inside and after the main loop.

Program 3: console_snake.c Here, the analysis shows that several variables can have multiple reaching definitions:

- **gameOver** has three reaching definitions: D1 (initial setup), D2 (wall collision), and D3 (tail collision). At the main game loop head (B1), all three may reach, reflecting the alternative execution paths where the game can either continue or terminate due to different collision types.
- **score** and **tailLength** are updated in the fruit collision block (B10). Their definitions (D4 and D5) reach subsequent blocks, but no redefinitions occur elsewhere. Thus, they each have a single reaching definition after their point of generation.

Overall, **gameOver** exhibits multiple possible reaching definitions at most program points, consistent with the branching nature of collision handling in the snake game.

0.5 Results and Discussion

The constructed Control Flow Graphs (CFGs) accurately represented all control structures present in the selected programs, including conditional branches, loop constructs, and multiple switch-case paths. Each CFG was verified to capture both sequential and non-sequential control flows, demonstrating the correctness of the graph-generation process.

The computed Cyclomatic Complexity (CC) values aligned with expectations based on the number of decision points and control paths in each program. Programs with more conditional branches and nested structures exhibited higher CC values, confirming the relationship between branching and program complexity.

During the Reaching Definitions Analysis (RDA), convergence was achieved within a few iterations (typically three), indicating that the implemented data-flow equations were stable and efficient. The iterative computation correctly identified all reaching definitions for each variable at every program point.

The analysis results also revealed variables with multiple possible reaching definitions at the same program point, such as **running**, **gameOver**, and **choice**. These instances correspond to program states influenced by user inputs or loop-control variables, highlighting the analyzer's ability to detect ambiguous variable states and potential redundant assignments.

Overall, the results demonstrate that the implemented analyzer effectively models real-world program behavior and can serve as a foundational tool for further static analysis, such as constant propagation or dead code elimination.

0.6 Conclusion

The Reaching Definitions Analyzer was successfully implemented and validated through three non-trivial C programs. Control Flow Graphs were automatically generated using Graphviz, and Cyclomatic Complexity metrics were computed to quantify structural complexity.

The data-flow analysis produced results consistent with theoretical expectations, achieving convergence efficiently and accurately identifying variables with multiple reaching definitions.

This experiment provided valuable insights into compiler optimization principles, particularly the role of data-flow analysis in detecting redundant computations and improving code efficiency. Through this lab, the understanding of static program analysis and its application to compiler design and optimization was significantly reinforced.

References

- Lecture 7 Slides — CS202: Software Tools and Techniques for CSE
- Github repo link https://github.com/kneeshit/STT_Assignments/tree/main/Lab7
- Wikipedia: Data-Flow Analysis
- Graphviz Documentation
- PyGraphviz Library